

3IRC – Mise en œuvre d'un système à microprocesseur

3 - Interruptions dans un 8051F020

Version: 23/02/2026 14:57



Interruptions

Définition et Généralités

Mise en place des interruptions

Au fait, c'est quoi une interruption?



Une interruption: pour quoi faire?

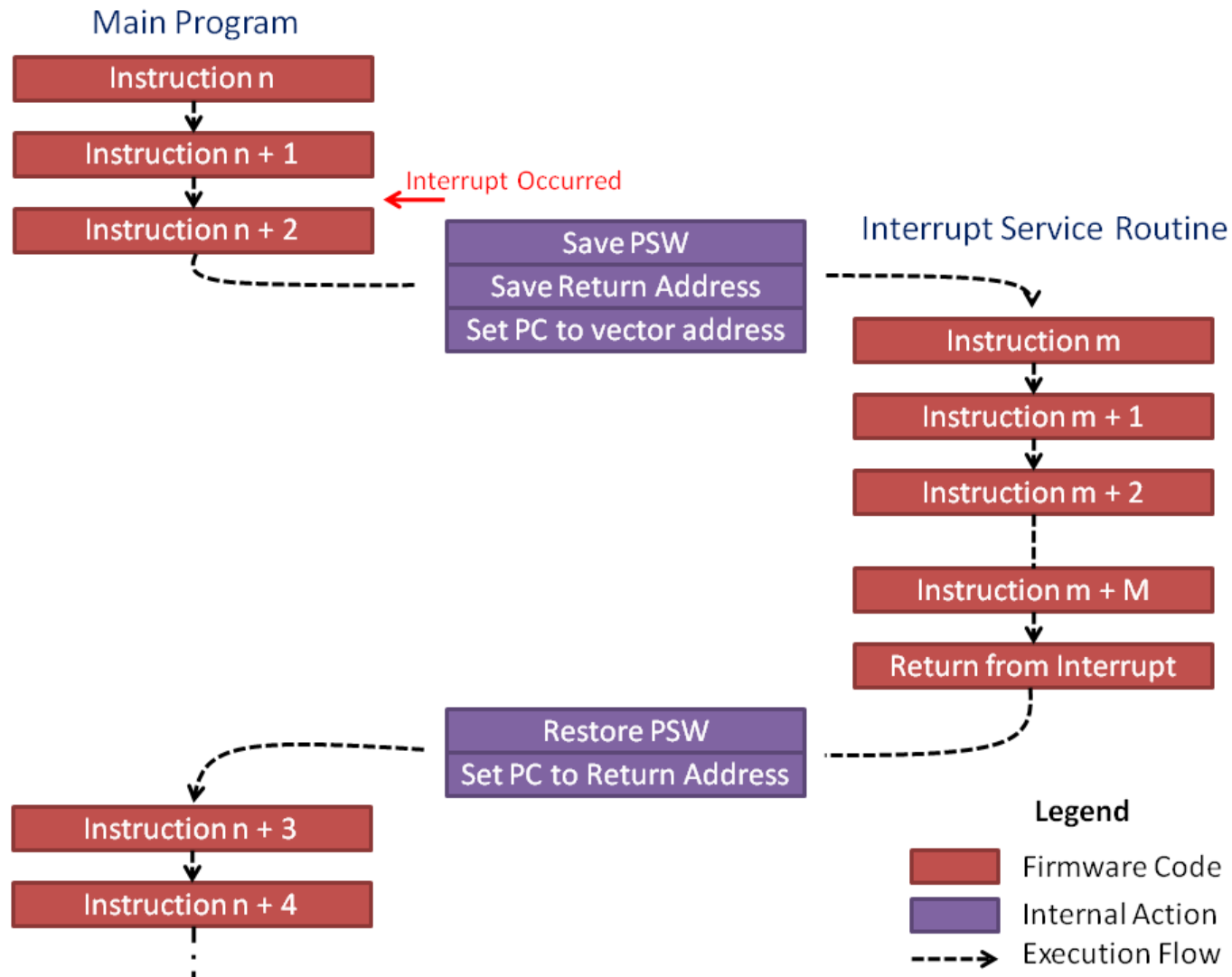
Définition: Une interruption désigne un mécanisme à la fois matériel et logiciel qui permet de suspendre temporairement l'exécution d'un programme pour exécuter un programme prioritaire chargé de traiter l'arrivée d'un évènement

- Les interruptions sont utilisées pour gérer des évènements externes **au processeur** (Central Processing Unit)
- Elles permettent d'éviter au processeur de scruter en permanence l'arrivée d'un évènement.
- Elles permettent au processeur d'exécuter de manière quasi immédiate du code pour traiter un évènement externe
- Dans le cas d'interruptions externes, elles permettent de capturer des évènements fugitifs



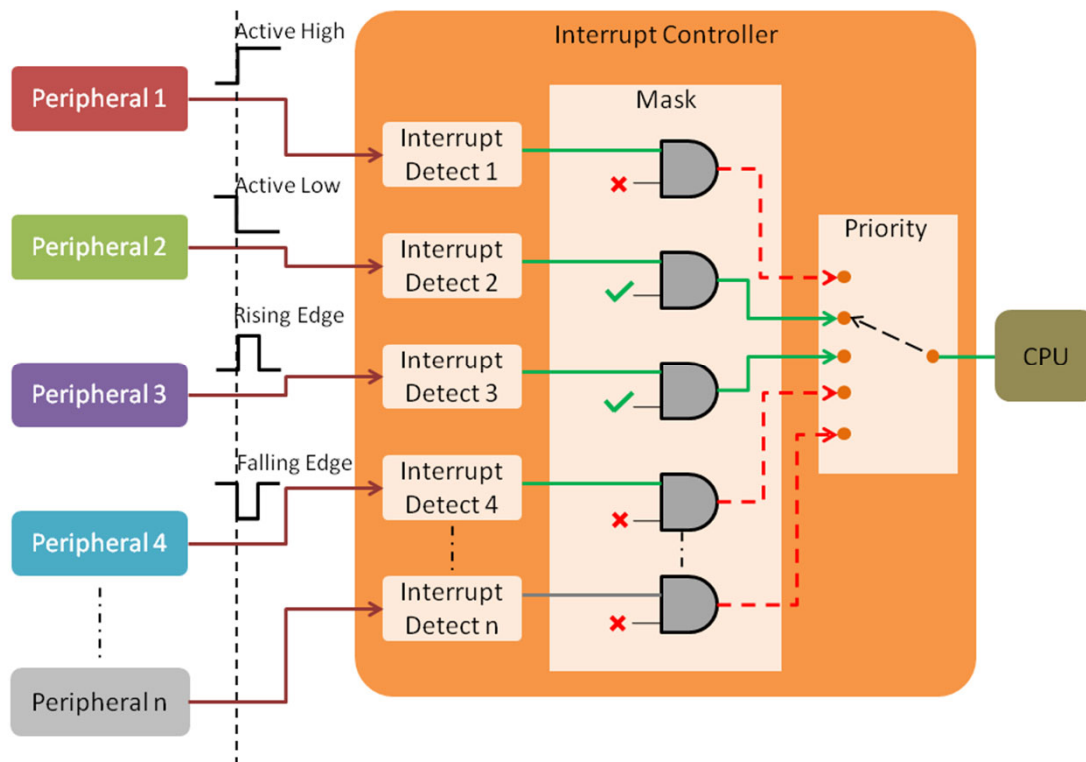
Attention, on parle ici d'évènement externes au processeur. Aussi dans un microcontrôleur les périphériques intégrés génèrent des évènement externes au processeur, et peuvent donc produire des interruptions.

Mécanisme d'interruption « classique » au niveau assembleur



Source: <http://www.embien.com/blog/interrupt-handling-in-embedded-software/>

Validation et priorisation des interruptions



1. Les interruptions peuvent en général être **masquées** (autorisées ou pas)
2. Les interruptions sont classées par **priorité** (2 niveaux sur le 8051F020)
3. Selon les priorités définies, une **interruption peut être interrompible**.
Une interruption de priorité basse pourra être interrompue par une interruption de priorité plus haute



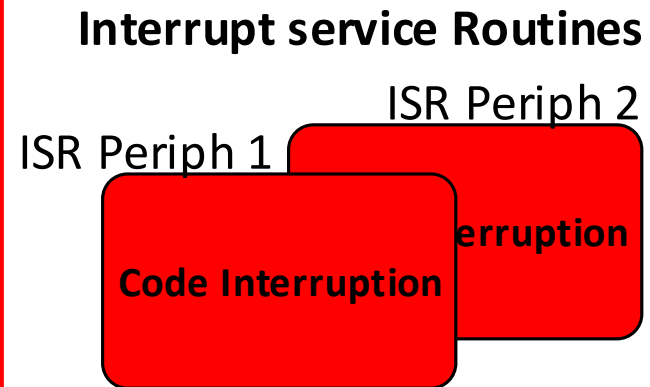
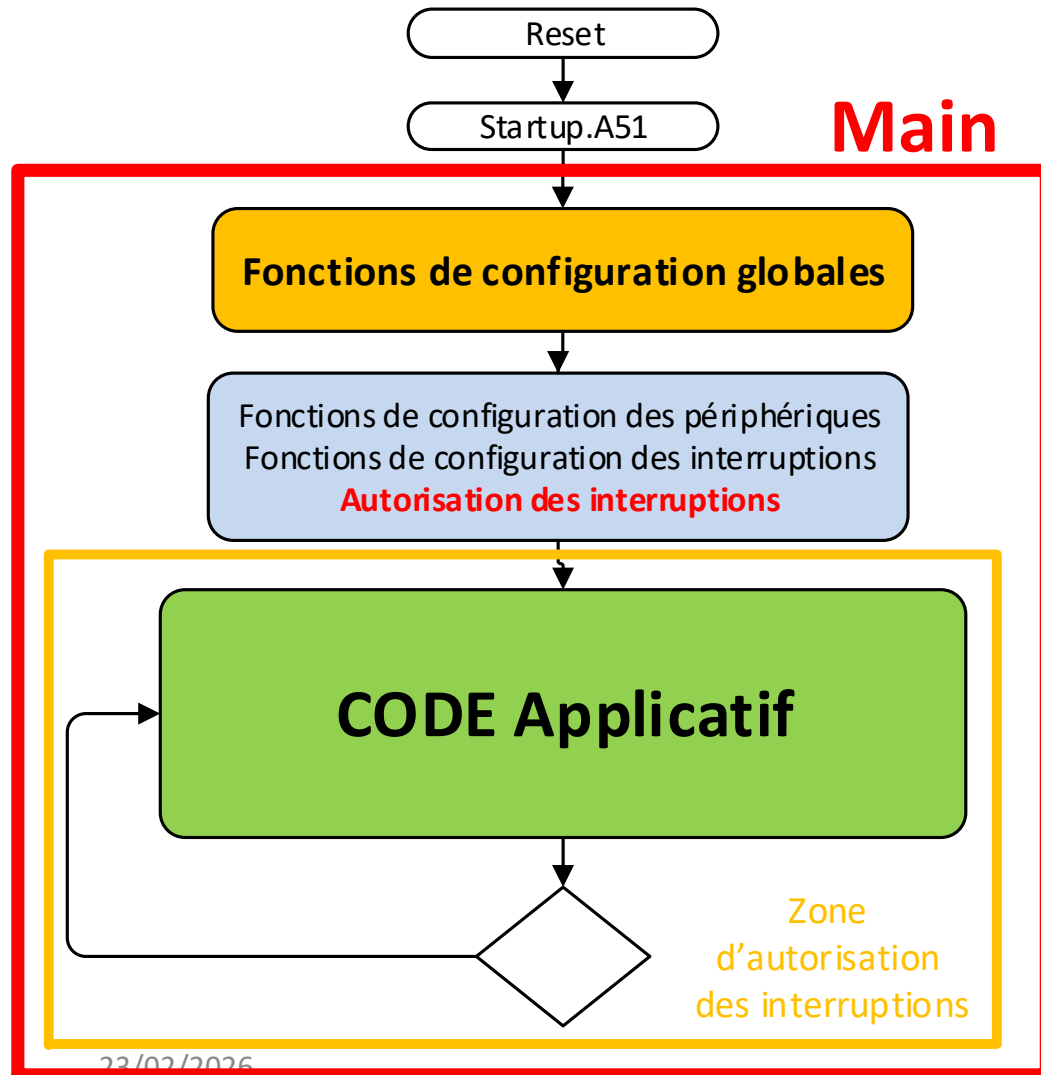
Règle de base: en général, un programme d'interruption se doit d'être le plus court possible.

Formulé autrement: il est impératif de bien maîtriser le temps d'exécution d'un programme d'interruption



Exécution des fonctions d'interruption

Les fonctions d'interruption sont déclenchées par un événement externe au processeur, et peuvent interrompre l'exécution de la fonction **Main** à tout instant



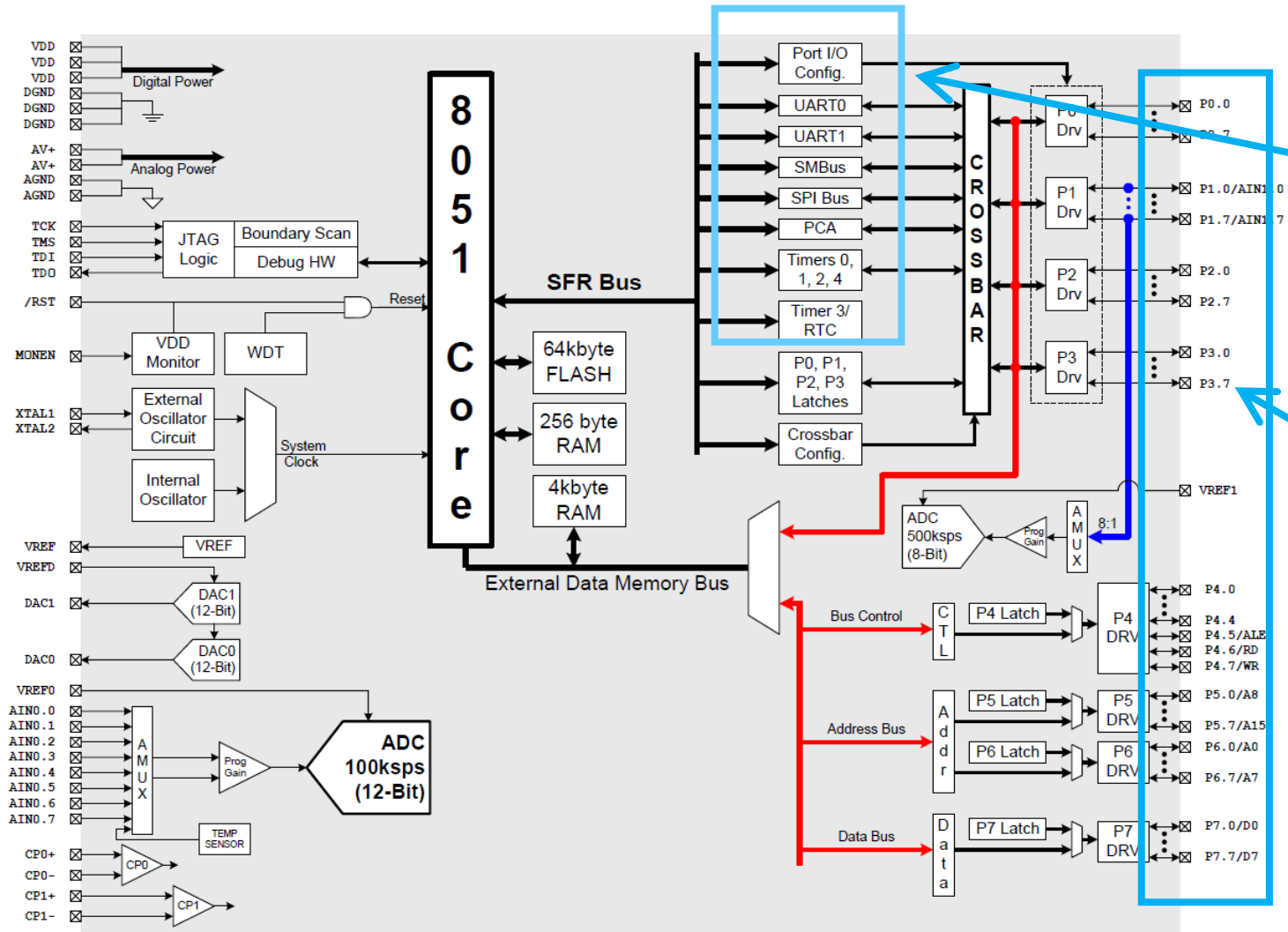
Exécution Code





Les Interruptions dans le 8051F020

Interruptions internes ou externe

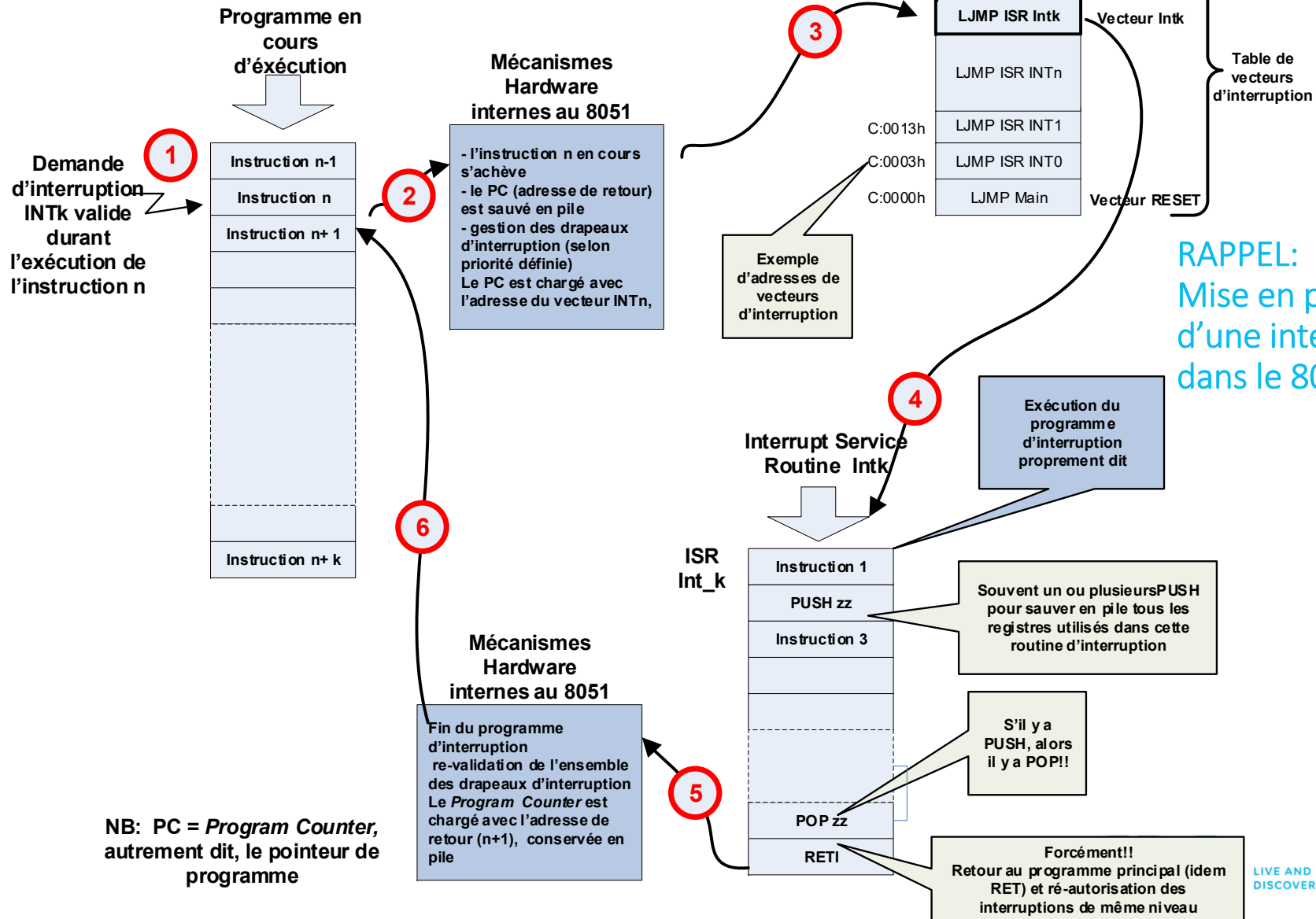


Sources d'interruptions internes: produites par les périphériques internes du microcontrôleur

! IMPORTANT

Sources d'interruptions externes: produites par des signaux extérieurs (associés à des événements) connectés sur des **entrées spécifiques** d'interruption

Cycle complet du déroulement d'une interruption dans un 8051



Utilisation des interruptions dans un 8051

L'utilisation des interruptions dans un processeur peut se résumer en 2 étapes distinctes:

- **Programmation de l'interruption.** Pour configurer et autoriser l'interruption. Ceci revient à lire et écrire dans des registres dédiés à la gestion des diverses interruptions.
 - Quel type d'évènement déclenchant (facultatif)
 - Autorisation de l'interruption
 - Priorisation de l'interruption
- **Utilisation de l'interruption.** Rédaction d'un code qui s'exécutera lors de l'interruption avec quelques obligations au niveau du code:
 - ✓ « Acquittement de l'interruption
 - ✓ Gestion de drapeaux matériels (usage debug)
 - ✓ Faire en sorte que ce code soit bien reconnu comme programme d'interruption pour une interruption donnée



Rappel: Procédure de mise en place d'une interruption en assembleur

1. Ecrire la routine d'interruption ISR (*Interrupt Service Routine*), à la manière d'un sous-programme. Par contre, cette routine doit obligatoirement se terminer par une instruction « **RETI** » au lieu de « **RET** ».
2. Placer dans la table de vecteurs d'interruption, à l'adresse réservée pour ce vecteur d'interruption, une ligne de code permettant le saut inconditionnel (type JMP) vers la routine ISR de traitement de l'interruption.
3. Configurer le périphérique pour qu'il soit en mesure de produire des demandes d'interruption au moment souhaité.
4. Configurer le bit (*Enable Flag*) dans un des registres de validation d'interruption (IE, EIE1 et EIE2) pour autoriser cette interruption.
5. Choisir le niveau de priorité donné à cette interruption par configuration du bit adéquat (*Priority Control*) dans un des registres de gestion des priorités (IP, EIP1 et EIP2).
6. Pour terminer, autoriser la prise en charge globale des interruptions en validant le bit EA du registre IE.

UTILISATION
de l'interruption

PROGRAMMATION
de l'interruption

Programmation des interruptions dans un 8051

Programmation de l'interruption.

Ceci revient à lire et écrire dans des registres dédiés à la gestion des diverses interruptions

Ne sont représentées ici que quelques une des sources possible d'interruption...

Signaux externes

Pending Flags

Enable Flags

Priority Control

23/02/2026

13

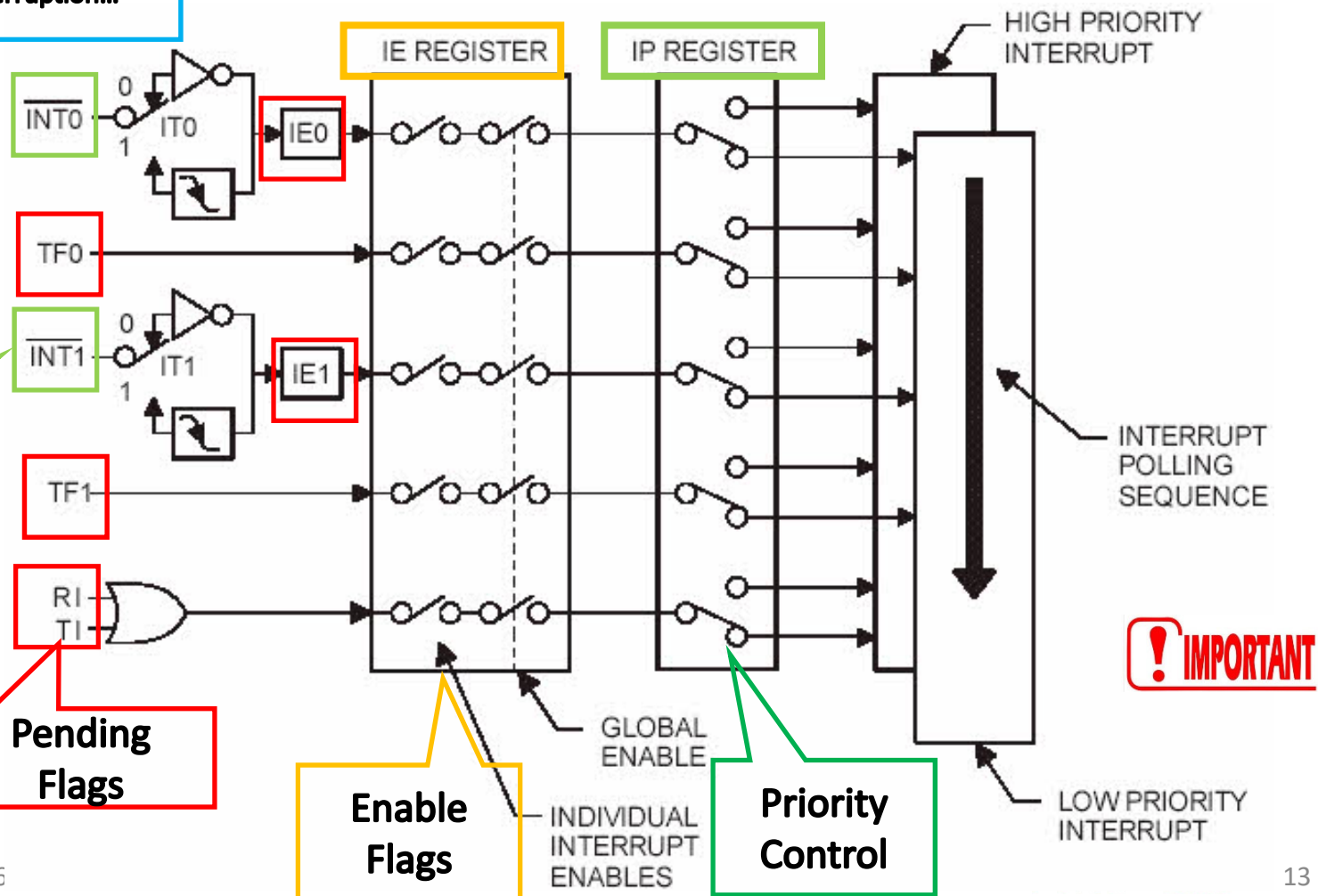


Table 12.4. Interrupt Summary

Interrupt Source	Interrupt Vector	Priority Order	Pending Flag	Bit addressable?	Cleared by HW?	Enable Flag	Priority Control
Reset	0x0000	Top	None	N/A	N/A	Always Enabled	Always Highest
External Interrupt 0 (/INT0)	0x0003	0	IE0 (TCON.1)	Y	Y	EX0 (IE.0)	PX0 (IP.0)
Timer 0 Overflow	0x000B	1	TF0 (TCON.5)	Y	Y	ET0 (IE.1)	PT0 (IP.1)
External Interrupt 1 (/INT1)	0x0013	2	IE1 (TCON.3)	Y	Y	EX1 (IE.2)	PX1 (IP.2)
Timer 1 Overflow	0x001B	3	TF1 (TCON.7)	Y	Y	ET1 (IE.3)	PT1 (IP.3)
UART0	0x0023	4	RI0 (SCON0.0) TI0 (SCON0.1)	Y		ES0 (IE.4)	PS0 (IP.4)
Timer 2 Overflow (or EXF2)	0x002B	5	TF2 (T2CON.7)	Y		ET2 (IE.5)	PT2 (IP.5)
Serial Peripheral Interface	0x0033	6	SPIF (SPI0CN.7)	Y		ESPI0 (EIE1.0)	PSPI0 (EIP1.0)
SMBus Interface	0x003B	7	SI (SMB0CN.3)	Y		ESMB0 (EIE1.1)	PSMB0 (EIP1.1)
ADC0 Window Comparator	0x0043	8	AD0WINT (ADC0CN.2)	Y		EWADC0 (EIE1.2)	PWADC0 (EIP1.2)
Programmable Counter Array	0x004B	9	CF (PCA0CN.7) CCFn (PCA0CN.n)	Y		EPCA0 (EIE1.3)	PPCA0 (EIP1.3)
Comparator 0 Falling Edge	0x0053	10	CP0FIF (CPT0CN.4)			ECP0F (EIE1.4)	PCP0F (EIP1.4)
Comparator 0 Rising Edge	0x005B	11	CP0RIF (CPT0CN.5)			ECP0R (EIE1.5)	PCP0R (EIP1.5)
Comparator 1 Falling Edge	0x0063	12	CP1FIF (CPT1CN.4)			ECPIF (EIE1.6)	PCPIF (EIP1.6)
Comparator 1 Rising Edge	0x006B	13	CP1RIF (CPT1CN.5)			ECPIR (EIE1.7)	PCPIR (EIP1.7)
Timer 3 Overflow	0x0073	14	TF3 (TMR3CN.7)			ET3 (EIE2.0)	PT3 (EIP2.0)
ADC0 End of Conversion	0x007B	15	AD0INT (ADC0CN.5)	Y		EADC0 (EIE2.1)	PADC0 (EIP2.1)
Timer 4 Overflow	0x0083	16	TF4 (T4CON.7)			ET4 (EIE2.2)	PT4 (EIP2.2)
ADC1 End of Conversion	0x008B	17	AD1INT (ADC1CN.5)			EADC1 (EIE2.3)	PADC1 (EIP2.3)
External Interrupt 6	0x0093	18	IE6 (P3IF.5)			EX6 (EIE2.4)	PX6 (EIP2.4)
External Interrupt 7	0x009B	19	IE7 (P3IF.6)			EX7 (EIE2.5)	PX7 (EIP2.5)
UART1	0x00A3	20	RI1 (SCON1.0) TI1 (SCON1.1)			ES1	PS1
External Crystal OSC Ready	0x00AB	21	XTLVLD (OSCCN.7)			EXVLD (EIE2.7)	PXVLD (EIP2.7)

Table des Interruptions dans le 8051F020

Éléments à manipuler dans les registres:

- **Priority Control:** Priorité registres IP + EIP1 + EIP2
- **Enable Flag:** Autorisation registres IE + EIE1 + EIE2
- **Pending Flags:** Drapeaux dans les Registres de périphériques

Définitions:

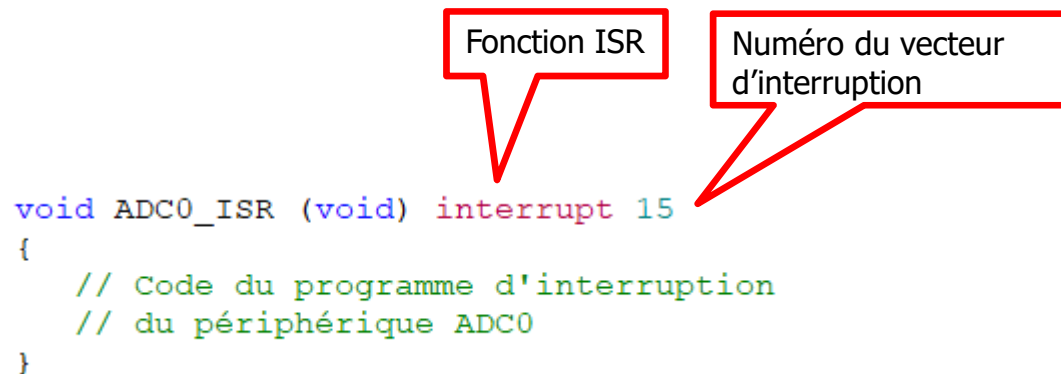
Pending Flag (Drapeau d'attente?): il signale l'arrivée d'un évènement (externe ou à l'intérieur d'un périphérique) **susceptible** de provoquer une interruption

Enable Flag (Drapeau d'autorisation), il autorise la transmission de la demande d'interruption au mécanisme matériel de gestion d'interruption.

Priority Control: fixe la priorité de cette interruption: prioritaire ou pas.

Utilisation d'une interruption - Mise en place d'une « Interrupt Service Routine (ISR) » en C Keil

Une fonction est identifiée comme fonction d'interruption, pour une interruption donnée, grâce au mot clé « Interrupt » suivi d'un numéro lors de la déclaration de la fonction



```
void ADC0_ISR (void) interrupt 15
{
    // Code du programme d'interruption
    // du périphérique ADC0
}
```

The diagram shows a C code snippet for an Interrupt Service Routine (ISR). A red box labeled 'Fonction ISR' points to the function name 'ADC0_ISR'. Another red box labeled 'Numéro du vecteur d'interruption' points to the 'interrupt 15' keyword in the function declaration.

Remarque 1: C'est le mot clé « **Interrupt** » qui informe le compilateur que cette fonction devra être traitée comme une fonction d'interruption. Le compilateur va donc devoir se charger de compléter la table de vecteurs d'interruption

Remarque 2: le numéro du vecteur d'interruption, permet au compilateur de déterminer l'adresse à partir de laquelle sera placé le code de saut vers la fonction d'interruption. **Ce numéro de vecteur d'interruption est donné dans la documentation constructeur (Table des interruptions, Colonne « Priority order »)**

Remarque 3: Le nom de la fonction peut être quelconque, mais l'usage veut que son nom indique clairement qu'il s'agit d'une fonction d'interruption.



Un exemple pour comprendre ce que fait le compilateur en présence d'une fonction déclarée comme fonction d'interruption:

Soit la fonction d'interruption suivante:

```
void ADC0_ISR (void) interrupt 15
{
    // Code du programme d'interruption
    // du périphérique ADC0
}
```

A la compilation, le compilateur détecte que la fonction ADC0_ISR doit être traitée comme une fonction d'interruption (grâce au mot clé « Interrupt »).

Timer 3 Overflow	0x0073	14	TF3 (TMR3CN.7)			ET3 (EIE2.0)	PT3 (EIP2.0)
ADC0 End of Conversion	0x007B	15	AD0INT (ADC0CN.5)	Y		EADC0 (EIE2.1)	PADC0 (EIP2.1)
Timer 4 Overflow	0x0083	16	TF4 (T4CON.7)			ET4 (EIE2.2)	PT4 (EIP2.2)

Grâce au numéro de vecteur d'interruption (15) dans la table de vecteurs d'interruptions, le compilateur détermine qu'il doit placer à l'adresse **0X007B**, un saut vers le sous-programme (la fonction) ADC0_ISR.

Enfin, le compilateur fait en sorte de terminer le sous-programme ADC0_ISR par l'instruction assembleur RETI au lieu d'un RET

Activité 1 – Analyse et mise en œuvre d'un code existant

Objectifs:

- Prise en main d'un code qui met en œuvre une interruption
- Essai du code sur la carte
- Analyse de la mise en place d'une interruption



Gestions des priorités dans les Interruptions

Cas du 8051F020

Interruptions dans le 8051F020

Priority Order: Numéro du vecteur d'interruption – utilisé par le compilateur pour déterminer l'adresse du vecteur d'interruption

Table 12.4. Interrupt Summary

Interrupt Source	Interrupt Vector	Priority Order	Pending Flag	Bit addressable?	Cleared by HW?	Enable Flag	Priority Control
Reset	0x0000	Top	None	N/A	N/A	Always Enabled	Always Highest
External Interrupt 0 (/INT0)	0x0003	0	IE0 (TCON.1)	Y	Y	EX0 (IE.0)	PX0 (IP.0)
Timer 0 Overflow	0x000B	1	TF0 (TCON.5)	Y	Y	ET0 (IE.1)	PT0 (IP.1)
External Interrupt 1 (/INT1)	0x0013	2	IE1 (TCON.3)	Y	Y	EX1 (IE.2)	PX1 (IP.2)
Timer 1 Overflow	0x001B	3	TF1 (TCON.7)	Y	Y	ET1 (IE.3)	PT1 (IP.3)
UART0	0x0023	4	RI0 (SCON0.0) TI0 (SCON0.1)	Y		ES0 (IE.4)	PS0 (IP.4)

Confusion « Priority Order » et « Priority Control »

Les interruptions dans les processeurs de la famille 8051 peuvent avoir **2 niveaux** de priorité: Haut et Bas. Ces niveaux de priorité sont fixés pour chaque interruption en agissant sur le **bit « Priority Control »** de chaque source d'interruption.

Les règles de priorité sont les suivantes:

- Une interruption de priorité Haute en cours d'exécution ne peut pas être interrompue (sauf par un Reset).
- Une interruption de priorité Basse, en cours d'exécution, peut être interrompue par une interruption de priorité haute. A l'issue de l'exécution du code de cette interruption de priorité Haute, l'interruption de priorité Basse pourra s'achever

Dans l'hypothèse où N interruptions de même niveau de priorité sont en attente d'être servies (par exemple durant l'exécution d'une autre interruption) alors la priorité sera donnée à celle dont le « priority order » est le plus faible.

Table 12.4. Interrupt Summary

Interrupt Source	Interrupt Vector	Priority Order	Pending Flag	Bit addressable?	Cleared by HW?	Enable Flag	Priority Control
Reset	0x0000	Top	None	N/A	N/A	Always Enabled	Always Highest

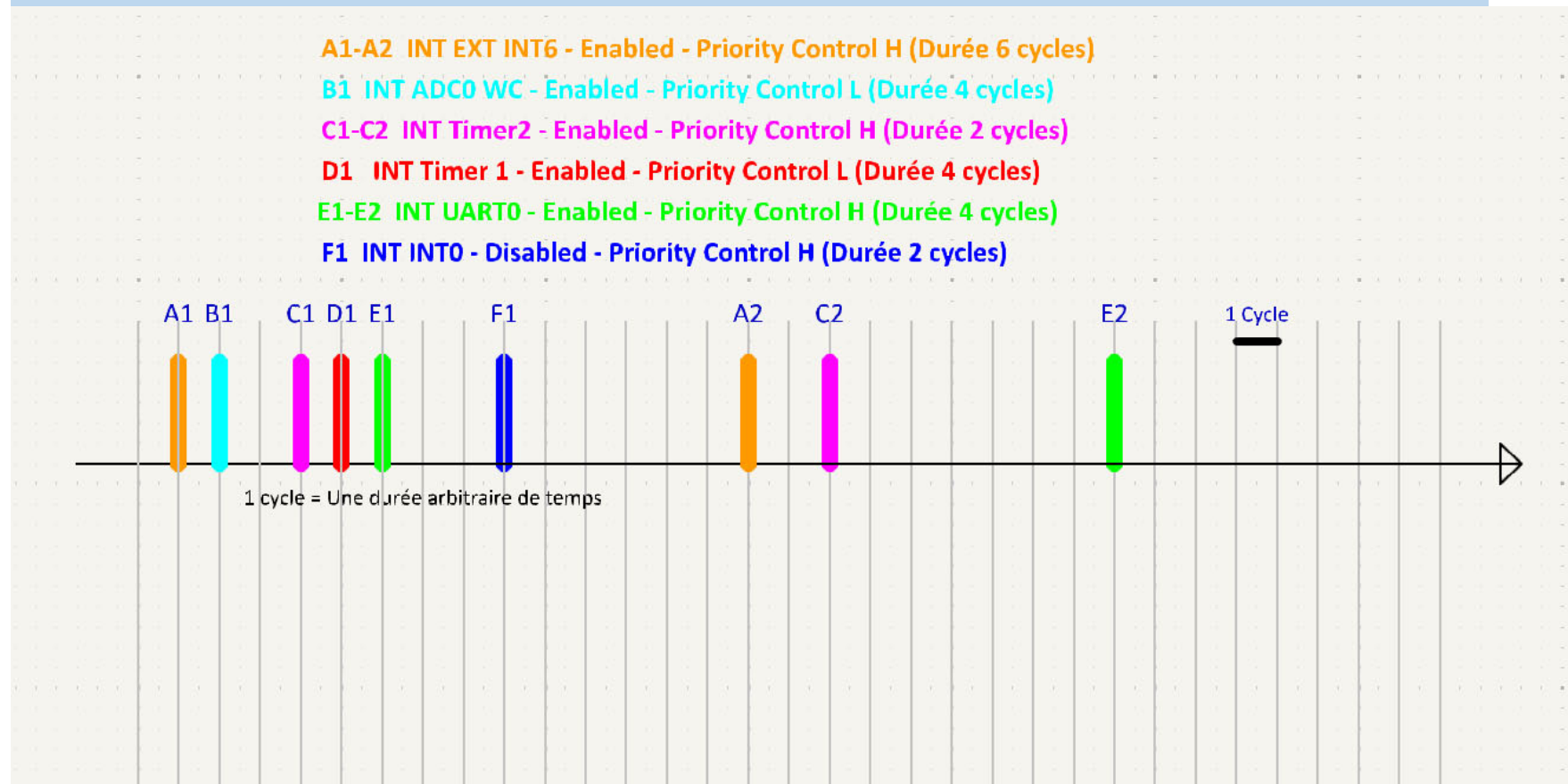
Interruptions internes et interruptions externes dans le 8051F020

- Rappel: une source d'interruption est dite « **interne** » lorsqu'elle est produite par un périphérique interne au microcontrôleur: par exemple les timers (0,1...4), les interfaces série UART, SPI, I2C et les convertisseurs analogiques/numériques peuvent générer des demandes d'interruption
- Rappel: une source d'interruption est dite « **externe** » lorsqu'elle est produite par un dispositif externe branché sur une broche du microcontrôleur, broche susceptible (par construction et configuration) de provoquer une demande d'interruption.
- Interruptions **externes disponibles** sur le 8051F020:
 - INT0 accessible sur une broche P0.0 à P2.3 (selon configuration de la matrice d'interconnexion Crossbar)
 - INT1 accessible sur une broche P0.0 à P2.5 (selon configuration de la matrice d'interconnexion Crossbar)
 - INT6 accessible sur la broche P3.6
 - INT7 accessible sur la broche P3.7

Activité 2 – Analyse de Priorité d'interruptions

Objectifs:

Dans le schéma ci-dessous, qui reproduit l'ordonnancement temporel des demandes d'interruption, déterminez l'ordre d'exécution des différents programme d'interruption compte tenu des priorités définies





Mise en œuvre détaillée d'une interruption dans le 8051F020

Utilisation de la documentation: Informations relatives aux registres

Chaque registre est décrit de manière synthétique dans un tableau récapitulant de nombreuses informations le concernant

Figure 22.5. TCON: Timer Control Register

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	Reset Value
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0	00000000
Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	SFR Address: 0x88
(bit addressable)								
Bit7:	TF1: Timer 1 Overflow Flag. Set by hardware when Timer 1 overflows. This flag can be cleared by software but is automatically cleared when the CPU vectors to the Timer 1 interrupt service routine. 0: No Timer 1 overflow detected. 1: Timer 1 has overflowed.							
Bit6:	TR1: Timer 1 Run Control. 0: Timer 1 disabled. 1: Timer 1 enabled.							
Bit5:	TF0: Timer 0 Overflow Flag. Set by hardware when Timer 0 overflows. This flag can be cleared by software but is automatically cleared when the CPU vectors to the Timer 0 interrupt service routine. 0: No Timer 0 overflow detected. 1: Timer 0 has overflowed.							

Accessibilité des bits en lecture ou en écriture

Indique si le registre est adressable bit à bit

Contenu du registre après Reset

Description du rôle de chaque bit

Règle de base pour l'utilisation des interruptions



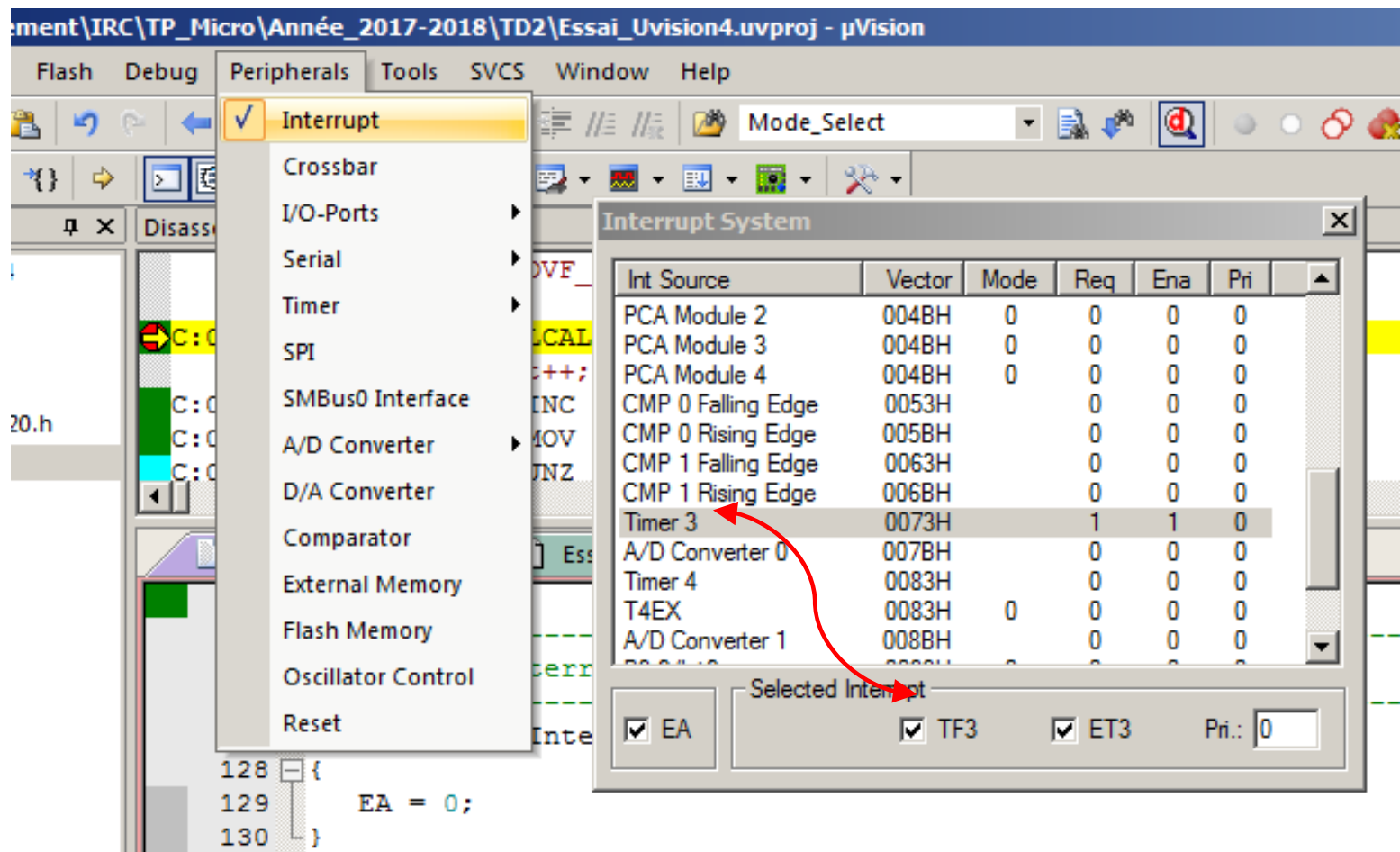
Avant d'autoriser une interruption pour un périphérique donné, on écrit d'abord le programme d'interruption correspondant!

Erreur classique: une interruption autorisée, mais pas de code d'interruption: plantage garanti!!



Débogueur et interruptions...

Le débogueur donne la possibilité de visualiser quelles sont les interrupteurs autorisés, quelles sont les demandes d'interruptions ainsi que des paramètres accessoires de configuration d'interruption.



Exemple de cas pratique – Mise en œuvre d’une interruption externe

Une application logicielle provoque (entre autre) une incrémentation toutes les 10 ms d’un compteur 10 bits (comptage de 0 à 1023). Par ailleurs, les 8 bits de poids fort de ce compteur sont envoyés sur le port 7.

On souhaite à l’aide d’un signal (SIG_IN) (actif au niveau H), externe au microcontrôleur, remettre à zéro la sortie le compteur.

Contrainte:

- La remise à zéro doit être effectuée le plus rapidement possible dès que le signal appelé SIG_IN passe au niveau haut.
- La durée du niveau haut de ce signal peut varier de 1 μ S à 1s.

23/02/2026



L'application logicielle

```
unsigned int compteur_logiciel=0;

while(1)
{
    // Application basique
    compteur_logiciel++;
    if (compteur_logiciel >= 1024) compteur_logiciel = 0;
    P7 = compteur_logiciel >>2; // envoi sur P7 des 8 bits de pds fort de compteur logiciel
    Delay(1); // 10ms
}
```

Comment gérer SIG_IN?

Est-il possible de gérer SIG_IN en scrutation? Quelles sont les limitations?

Quel est le temps de réponse à un évènement SIG_IN?

Peut-on capturer un évènement SIG_IN fugitif (10 micro-secondes)?

Utiliser des Interruptions?

En quoi une interruption résout les 2 limitations précédentes?

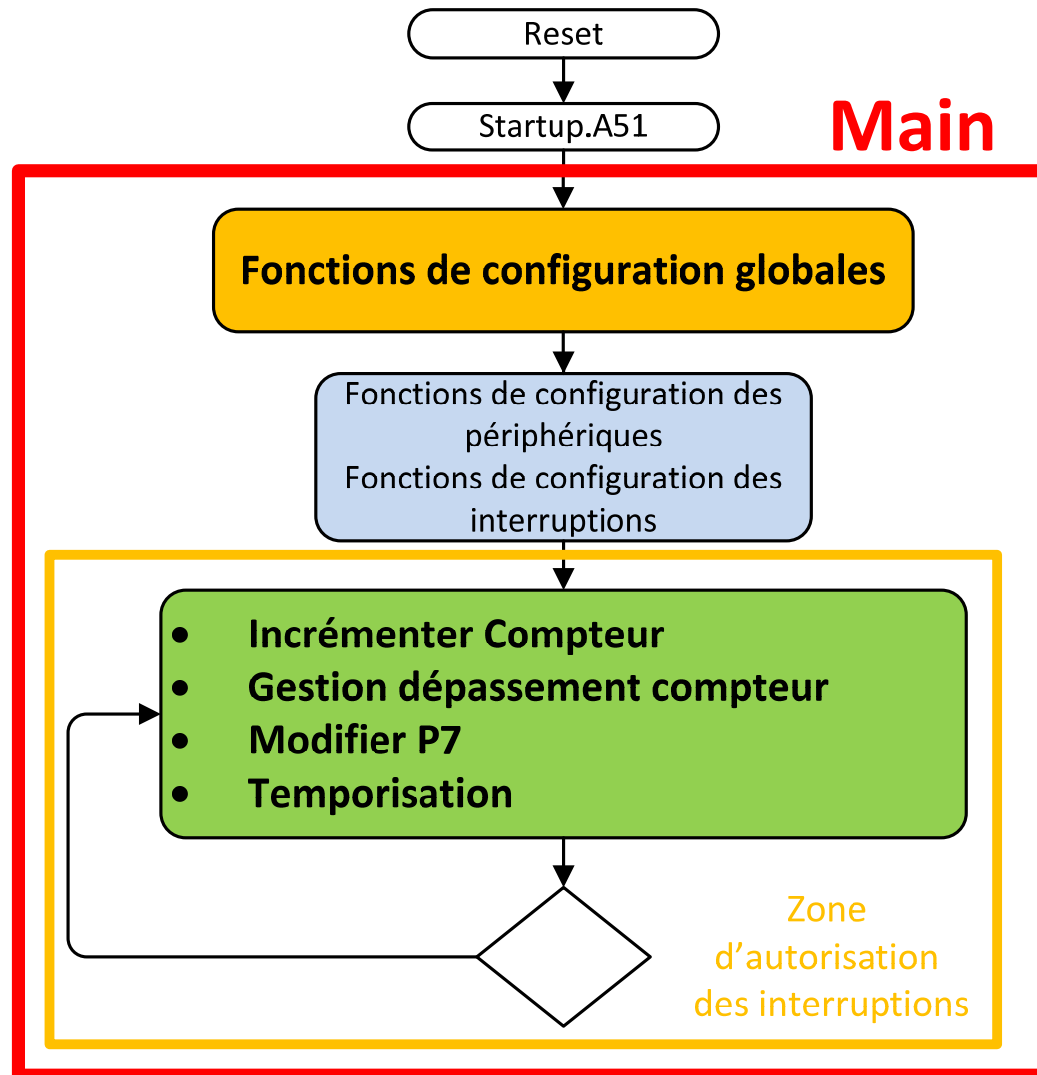
Quel signal d'interruption? C'est le passage au niveau haut de SIG_IN qui va déclencher une interruption.

Quelle Interruption? SIG_IN est extérieur au microcontrôleur, on va donc devoir mettre en œuvre une interruption externe

Mode de déclenchement? Idéalement on souhaiterait que le front montant de SIG_IN provoque une interruption

Quelle configuration?

Mode d'utilisation de l'interruption



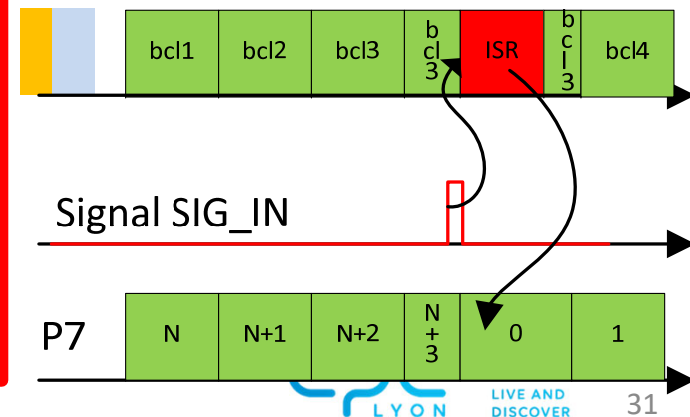
Main

Interrupt Service Routine
Source de déclenchement:
Front montant sur le signal SIG_IN

ISR

- Remise à zéro Compteur
- Ecriture dans P7

Exécution Code



La solution Interruption avec sources de déclenchement externe

L'objectif est de câbler le signal SIG_IN sur une broche d'interruption du processeur.

On dispose sur le 8051F020 de 4 entrées d'interruption externes:

- INT0, INT1, sensibles sur niveau bas ou sur front descendant (configurable). L'affectation sur les broches de INT0 et INT1 est géré via une matrice d'interconnexion (CROSSBAR).
- INT6 et INT7, sensible sur front montant ou descendant (configurable)
Par construction INT6 est reliée à P3.6 et INT7 à P3.7.

Compte tenu du dispositif (impulsion positive sur SIG_IN), on va utiliser une entrée d'interruption sensible sur front montant: INT6 est choisie (INT7 est aussi utilisable)

Pour résumer: à chaque front montant sur le signal SIG_IN, on va déclencher l'exécution d'un programme d'interruption INT6

INT6 dans la table de vecteur d'interruption

Table 12.4. Interrupt Summary

Interrupt Source	Interrupt Vector	Priority Order	Pending Flag	Bit addressable?	Cleared by HW?	Enable Flag	Priority Control
ADC1 End of Conversion	0x008B	17	AD1INT (ADC1CN.5)			EADC1 (EIE2.3)	PADC1 (EIP2.3)
External Interrupt 6	0x0093	18	IE6 (P3IF.5)			EX6 (EIE2.4)	PX6 (EIP2.4)
External Interrupt 7	0x009B	19	IE7 (P3IF.6)			EX7 (EIE2.5)	PX7 (EIP2.5)

12.3.2. External Interrupts

The remaining 2 external interrupts (External Interrupts 6-7) are edge-sensitive inputs configurable as active-low or active-high. The interrupt-pending flags and configuration bits for these interrupts are in the Port 3 Interrupt Flag Register shown in **Figure “17.19 P3IF: Port3 Interrupt Flag Register” on page 177.**

EXEMPLE Interruption INT6: Registre d'état, de validation, de priorité et de configuration

Registre EIE2

Figure 12.12. EIE2: Extended Interrupt Enable 2

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	Reset Value
EXVLD	ES1	EX7	EX6	EADC1	ET4	EADC0	ET3	00000000
Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	SFR Address: 0xE7

Bit7: EXVLD: Enable External Clock Source Valid (XTLVLD) Interrupt. This bit sets the masking of the XTLVLD interrupt.
 0: Disable XTLVLD interrupt.
 1: Enable interrupt requests generated by the XTLVLD flag (OSC_XCN.7)
 Bit6: ES1: Enable UART1 Interrupt. This bit sets the masking of the UART1 interrupt.
 0: Disable UART1 interrupt.
 1: Enable UART1 interrupt.
 Bit5: EX7: Enable External Interrupt 7. This bit sets the masking of External Interrupt 7.
 0: Disable External Interrupt 7.
 1: Enable interrupt requests generated by the External Interrupt 7 input pin.
 Bit4: EX6: Enable External Interrupt 6. This bit sets the masking of External Interrupt 6.
 0: Disable External Interrupt 6.
 1: Enable interrupt requests generated by the External Interrupt 6 input pin.

Registre EIP2

Figure 12.14. EIP2: Extended Interrupt Priority 2

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
PXVLD	EP1	PX7	PX6	PADC1	PT4	PADC0	PT3	PT2
Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	Bit0

Bit7: PXVLD: External Clock Source Valid (XTLVLD) Interrupt Priority Control. This bit sets the priority of the XTLVLD interrupt.
 0: XTLVLD interrupt set to low priority level.
 1: XTLVLD interrupt set to high priority level.
 Bit6: EP1: UART1 Interrupt Priority Control. This bit sets the priority of the UART1 interrupt.
 0: UART1 interrupt set to low priority.
 1: UART1 interrupt set to high priority.
 Bit5: PX7: External Interrupt 7 Priority Control. This bit sets the priority of the External Interrupt 7.
 0: External Interrupt 7 set to low priority level.
 1: External Interrupt 7 set to high priority level.
 Bit4: PX6: External Interrupt 6 Priority Control. This bit sets the priority of the External Interrupt 6.
 0: External Interrupt 6 set to low priority level.
 1: External Interrupt 6 set to high priority level.

Figure 17.19. P3IF: Port3 Interrupt Flag Register

Registre P3IF

R/W	R/W	R	R	R/W	R/W	R/W	R/W	Reset Value
IE7	IE6	-	-	IE7CF	IE6CF	-	-	00000000
Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	SFR Address: 0xAD

Bit7: IE7: External Interrupt 7 Pending Flag
 0: No falling edge has been detected on P3.7 since this bit was last cleared.
 1: This flag is set by hardware when a falling edge on P3.7 is detected.
 Bit6: IE6: External Interrupt 6 Pending Flag
 0: No falling edge has been detected on P3.6 since this bit was last cleared.
 1: This flag is set by hardware when a falling edge on P3.6 is detected.
 Bit5: IE5: External Interrupt 5 Pending Flag
 0: No falling edge has been detected on P3.5 since this bit was last cleared.
 1: This flag is set by hardware when a falling edge on P3.5 is detected.
 Bit4: IE4: External Interrupt 4 Pending Flag
 0: No falling edge has been detected on P3.4 since this bit was last cleared.
 1: This flag is set by hardware when a falling edge on P3.4 is detected.
 Bit3: IE7CF: External Interrupt 7 Edge Configuration
 0: External Interrupt 7 triggered by a falling edge on the IE7 input.
 1: External Interrupt 7 triggered by a rising edge on the IE7 input.
 Bit2: IE6CF: External Interrupt 6 Edge Configuration
 0: External Interrupt 6 triggered by a falling edge on the IE6 input.
 1: External Interrupt 6 triggered by a rising edge on the IE6 input.
 Bit1: IE5CF: External Interrupt 5 Edge Configuration
 0: External Interrupt 5 triggered by a falling edge on the IE5 input.
 1: External Interrupt 5 triggered by a rising edge on the IE5 input.
 Bit0: IE4CF: External Interrupt 4 Edge Configuration
 0: External Interrupt 4 triggered by a falling edge on the IE4 input.
 1: External Interrupt 4 triggered by a rising edge on the IE4 input.

Figure 12.9. IE: Interrupt Enable

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	Reset Value
EA	IEGF0	ET2	ES0	ET1	EX1	ET0	EX0	00000000
Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	SFR Address: 0xA8

Bit7: EA: Enable All Interrupts. This bit globally enables/disables all interrupts. When set to '0', individual interrupt mask settings are overridden.
 0: Disable all interrupt sources.
 1: Enable each interrupt according to its individual mask settings.
 Bit6: IEGF0: General Purpose Flag 0.

Rappel: Procédure de mise en place d'une interruption – Etapes 1-2

Etapes 1-2 - Utilisation

1. Ecrire la routine d'interruption ISR (*Interrupt Service Routine*), à la manière d'un sous-programme. Par contre, cette routine doit obligatoirement se terminer par une instruction « **RETI** » au lieu de « **RET** ».
2. Placer dans la table de vecteurs d'interruption, à l'adresse réservée pour ce vecteur d'interruption, une ligne de code permettant le saut inconditionnel (type JMP) vers la routine ISR de traitement de l'interruption.

En C, c'est le mot clé « **Interrupt** » suivi du **numéro de vecteur**, qui permet au compilateur de gérer ces 2 étapes

3. Configurer le périphérique pour qu'il soit en mesure de produire des demandes d'interruption au moment souhaité.
4. Configurer le bit (*Enable Flag*) dans un des registres de validation d'interruption (IE, EIE1 et EIE2) pour autoriser cette interruption.
5. Choisir le niveau de priorité donné à cette interruption par configuration du bit adéquat (*Priority Control*) dans un des registres de gestion des priorités (IP, EIP1 et EIP2).
6. Pour terminer, autoriser la prise en charge globale des interruptions en validant le bit EA du registre IE.

Etapes 1-2 – Ecriture d'un squelette de fonction d'interruption

Ecriture d'un squelette de fonction d'interruption externe INT6

```
void ISR_INT6() interrupt 18
{
}

```

Le nom de la fonction d'interruption peut être quelconque.

Mais par soucis de clarté, l'usage est d'utiliser le préfixe « ISR » « **Interrupt Service Routine** » et un suffixe correspondant au nom de l'interruption (ici « INT6 »)

C'est ce numéro qui informe le compilateur que cette fonction sera codée comme fonction d'interruption INT6

ADC1 End of Conversion	0x008B	17	AD11IN1 (ADC1CN.5)			EADC1 (EIE2.3)	PADC1 (EIP2.3)
External Interrupt 6	0x0093	18	IE6 (P3IF.5)			EX6 (EIE2.4)	PX6 (EIP2.4)
External Interrupt 7	0x009B	19	IE7 (P3IF.6)			EX7 (EIE2.5)	PX7 (EIP2.5)

Etapes 1-2 – Éléments complémentaires dans une fonction d'interruption

Toute fonction d'interruption, voit son déclenchement conditionné par un drapeau nommé « Pending Flag ». C'est le passage à 1 de ce dernier qui déclenche la demande d'interruption. L'interruption est ensuite traitée si elle est autorisée.

En général, ce Pending Flag n'est pas automatiquement remis à zéro. Il est donc indispensable de le remettre à zéro par logiciel. Cette remise à zéro sera réalisée dans le programme d'interruption.

```
void ISR_INT6() interrupt 18
```

```
{  
    P3IF &=~ (1<<6) ;           // Clear FLAG IE6  
}
```

Figure 17.19. P3IF: Port3 Interrupt Flag Register

R/W	R/W	R	R	R/W	R/W	R/W	R/W
IE7	IE6	-	-	IE7CF	IE6CF	-	-
Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0

Bit7:	IE7: External Interrupt 7 Pending Flag 0: No falling edge has been detected on P3.7 since this bit was last cleared 1: This flag is set by hardware when a falling edge on P3.7 is detected.
Bit6:	IE6: External Interrupt 6 Pending Flag 0: No falling edge has been detected on P3.6 since this bit was last cleared. 1: This flag is set by hardware when a falling edge on P3.6 is detected.

Erreur dans la doc:
Il faudrait plutôt lire:
« ... when a falling or rising edge
(according to IE6CF) on P3.6 is
detected

Etapes 1-2 – Éléments accessoires utiles dans une fonction d'interruption

Il est souvent pratique de pouvoir visualiser sur un oscilloscope l'exécution d'un programme d'interruption (a-t-on une interruption? Quelle est sa récurrence? Sa durée?), aussi, la mise en œuvre d'une broche d'un port utilisée en GPIO permet cette visualisation.

```
sbit VISU_Temoin_INT6 = P...
```

```
void ISR_INT6() interrupt 18
```

```
{
```

```
    VISU_Temoin_INT6 = 1; // Facultatif - Signal Témoin INT6 pour  
                           // visualisation sur oscilloscope
```

```
    P3IF &=~0x40;        // Clear FLAG IE6
```

```
    VISU_Temoin_INT6 = 0;
```

```
}
```

Rappel: Procédure de mise en place d'une interruption – Etape 3

1. Ecrire la routine d'interruption ISR (*Interrupt Service Routine*), à la manière d'un sous-programme. Par contre, cette routine doit obligatoirement se terminer par une instruction « **RETI** » au lieu de « **RET** ».
2. Placer dans la table de vecteurs d'interruption, à l'adresse réservée pour ce vecteur d'interruption, une ligne de code permettant le saut inconditionnel (type JMP) vers la routine ISR de traitement de l'interruption.

Etape 3 - Configuration

3. Configurer le périphérique pour qu'il soit en mesure de produire des demandes d'interruption au moment souhaité.
4. Configurer le bit (*Enable Flag*) dans un des registres de validation d'interruption (IE, EIE1 et EIE2) pour autoriser cette interruption.
5. Choisir le niveau de priorité donné à cette interruption par configuration du bit adéquat (*Priority Control*) dans un des registres de gestion des priorités (IP, EIP1 et EIP2).
6. Pour terminer, autoriser la prise en charge globale des interruptions en validant le bit EA du registre IE.

Etape 3 – Configuration de l'interruption

Ecriture d'une fonction de configuration de l'interruption.

Configuration du « périphérique » pour être en mesure de produire des demandes d'interruption et/ou spécification mode de déclenchement de l'interruption → Action sur le registre P3IF

Au passage, remise à zéro du « Pending flag »

```
void Config_INT6_Ext(void)
{
    P3IF &= ~(1<<6); // Pending flag IE6 remis à zéro (au cas où...)
    P3IF |= (1<<2); // IE7CF=1 -- Déclenchement INT6 sur front montant
}
```

Figure 17.19. P3IF: Port3 Interrupt Flag Register

R/W	R/W	R	R	R/W	R/W	R/W	R/W	Reset Value
IE7	IE6	-	-	IE7CF	IE6CF	-	-	00000000
Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	SFR Address: 0xAD
Bit7: IE7: External Interrupt 7 Pending Flag								
0: No falling edge has been detected on P3.7 since this bit was last cleared.								
1: This flag is set by hardware when a falling edge on P3.7 is detected.								
Bit6: IE6: External Interrupt 6 Pending Flag								
0: No falling edge has been detected on P3.6 since this bit was last cleared.								
1: This flag is set by hardware when a falling edge on P3.6 is detected.								
Bits5-4: UNUSED. Read = 00b, Write = don't care.								
Bit3: IE7CF: External Interrupt 7 Edge Configuration								
0: External Interrupt 7 triggered by a falling edge on the IE7 input.								
1: External Interrupt 7 triggered by a rising edge on the IE7 input.								
Bit2: IE6CF: External Interrupt 6 Edge Configuration								
0: External interrupt 6 triggered by a falling edge on the IE6 input.								
1: External Interrupt 6 triggered by a rising edge on the IE6 input.								

Rappel: Procédure de mise en place d'une interruption – Etape 4

1. Ecrire la routine d'interruption ISR (*Interrupt Service Routine*), à la manière d'un sous-programme. Par contre, cette routine doit obligatoirement se terminer par une instruction « **RETI** » au lieu de « **RET** ».
2. Placer dans la table de vecteurs d'interruption, à l'adresse réservée pour ce vecteur d'interruption, une ligne de code permettant le saut inconditionnel (type JMP) vers la routine ISR de traitement de l'interruption.
3. Configurer le périphérique pour qu'il soit en mesure de produire des demandes d'interruption au moment souhaité.

Etape 4 - Configuration

4. Configurer le bit (*Enable Flag*) dans un des registres de validation d'interruption (IE, EIE1 et EIE2) pour autoriser cette interruption.
5. Choisir le niveau de priorité donné à cette interruption par configuration du bit adéquat (*Priority Control*) dans un des registres de gestion des priorités (IP, EIP1 et EIP2).
6. Pour terminer, autoriser la prise en charge globale des interruptions en validant le bit EA du registre IE.

Etape 4 – Autorisation de l'interruption INT6

Toujours dans la fonction de configuration de l'interruption, modification d'un registre d'autorisation d'interruption pour autoriser INT6

```
void Config_INT6_Ext(void)
```

```
{  P3IF  &= ~(1<<6); // RAZ INT6 Pending Flag (P3IF page 177)

    P3IF |= (1<<2); // Déclenchement INT6 sur front montant

    EIE2 |= (1<<4); //EX6 = 1 Autorisation de l'interruption INT6 }
```

Figure 12.12. EIE2: Extended Interrupt Enable 2

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	Reset Value
EXVLD	ES1	EX7	EX6	EADC1	ET4	EADC0	ET3	00000000
Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	SFR Address: 0xE7

Bit4: EX6: Enable External Interrupt 6.
This bit sets the masking of External Interrupt 6.
0: Disable External Interrupt 6.
1: Enable interrupt requests generated by the External Interrupt 6 input pin.

Rappel: Procédure de mise en place d'une interruption – Etape 5

1. Ecrire la routine d'interruption ISR (*Interrupt Service Routine*), à la manière d'un sous-programme. Par contre, cette routine doit obligatoirement se terminer par une instruction « **RETI** » au lieu de « **RET** ».
2. Placer dans la table de vecteurs d'interruption, à l'adresse réservée pour ce vecteur d'interruption, une ligne de code permettant le saut inconditionnel (type JMP) vers la routine ISR de traitement de l'interruption.
3. Configurer le périphérique pour qu'il soit en mesure de produire des demandes d'interruption au moment souhaité.
4. Configurer le bit (*Enable Flag*) dans un des registres de validation d'interruption (IE, EIE1 et EIE2) pour autoriser cette interruption.
5. Choisir le niveau de priorité donné à cette interruption par configuration du bit adéquat (*Priority Control*) dans un des registres de gestion des priorités (IP, EIP1 et EIP2).
6. Pour terminer, autoriser la prise en charge globale des interruptions en validant le bit EA du registre IE.

Etape 5 - Configuration

Etape 5 - Gestion de la priorité de l'interruption

Gestion de la priorité de l'interruption (par rapport aux autres programmes d'interruptions) de INT6; Cette opération est réalisée dans la fonction de configuration de l'interruption

```
void Config_INT6_Ext(void)
{ P3IF &= ~(1<<6); // RAZ INT6 Pending Flag (P3IF page 177)

  P3IF |= (1<<2); // Déclenchement INT6 sur front montant

  EIE2 |= (1<<4); //EX6 = 1 Autorisation de l'interruption INT6

  EIP2 |= (1<<4); // Facultatif - Priorité haute
    pour l'interruption INT6 }
```

Figure 12.14. EIP2: Extended Interrupt Priority 2

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	Reset Value
PXVLD	EP1	PX7	PX6	PADC1	PT4	PADC0	PT3	00000000
Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	SFR Address:
Bit4: PX6: External Interrupt 6 Priority Control. This bit sets the priority of the External Interrupt 6. 0: External Interrupt 6 set to low priority level. 1: External Interrupt 6 set to high priority level.								

Rappel: Procédure de mise en place d'une interruption en assembleur – Etape 6

1. Ecrire la routine d'interruption ISR (*Interrupt Service Routine*), à la manière d'un sous-programme. Par contre, cette routine doit obligatoirement se terminer par une instruction « **RETI** » au lieu de « **RET** ».
2. Placer dans la table de vecteurs d'interruption, à l'adresse réservée pour ce vecteur d'interruption, une ligne de code permettant le saut inconditionnel (type JMP) vers la routine ISR de traitement de l'interruption.
3. Configurer le périphérique pour qu'il soit en mesure de produire des demandes d'interruption au moment souhaité.
4. Configurer le bit (*Enable Flag*) dans un des registres de validation d'interruption (IE, EIE1 et EIE2) pour autoriser cette interruption.
5. Choisir le niveau de priorité donné à cette interruption par configuration du bit adéquat (*Priority Control*) dans un des registres de gestion des priorités (IP, EIP1 et EIP2).
6. Pour terminer, autoriser la prise en charge globale des interruptions en validant le bit EA du registre IE.

Etape 6 - Configuration

Etape 6 – Autorisation « globale » des interruptions

Le bit EA mis à 1 permet d'autoriser toutes les interruptions **autorisées localement**

Le bit EA mis à 0, interdit toutes les interruptions quelles que soient les autorisations locales.

Cette validation globale est gérée au niveau de l'application globale: la fonction main en général.

```
48 //-----
49 void main (void) {
50
51 //*****
52 // CONFIGURATIONS GLOBALES
53     Init_Device();
54 //*****
55 // CONFIGURATION des périphériques
56
57     Config_INT_Ext();
58 //*****
59 // Séquence Démarrage
60     Phase_Demarrage();
61     EA = 1;
62 //*****
63 //Séquence usage
64     while(1)
65     {
66 //         Phase Usage();
```

Code applicatif dans le programme d'interruption INT6

Le code de l'interruption INT6

```
/** *****  
void ISR_INT6() interrupt 18  
{  
    VISU_Temoin_INT6 = 1; // Facultatif - Signal Témoin INT6 pour vérification  
                           // sur oscilloscope  
    P3IF &=~0x40;         // Clear FLAG IE6  
  
    compteur_logiciel = 0;  
    P7 = compteur_logiciel >>2;  
  
    VISU_Temoin_INT6 = 0;  
}
```

Contraintes sur la fonction `Main`

Attention à la gestion des variables partagées qui sont bien dans le `main` que dans le programme d'interruption.

Aussi, on dévalidera temporairement les interruptions afin d'éviter le déclenchement d'une interruption au milieu de la manipulation d'une variable partagée.

```
while(1)
{
    EA = 0; // Section de code critique
    compteur_logiciel++;
    if (compteur_logiciel >= 1024) compteur_logiciel = 0;
    P7 = compteur_logiciel >>2;
    EA = 1; // Fin Section de code critique
    Delay(1);
}
```

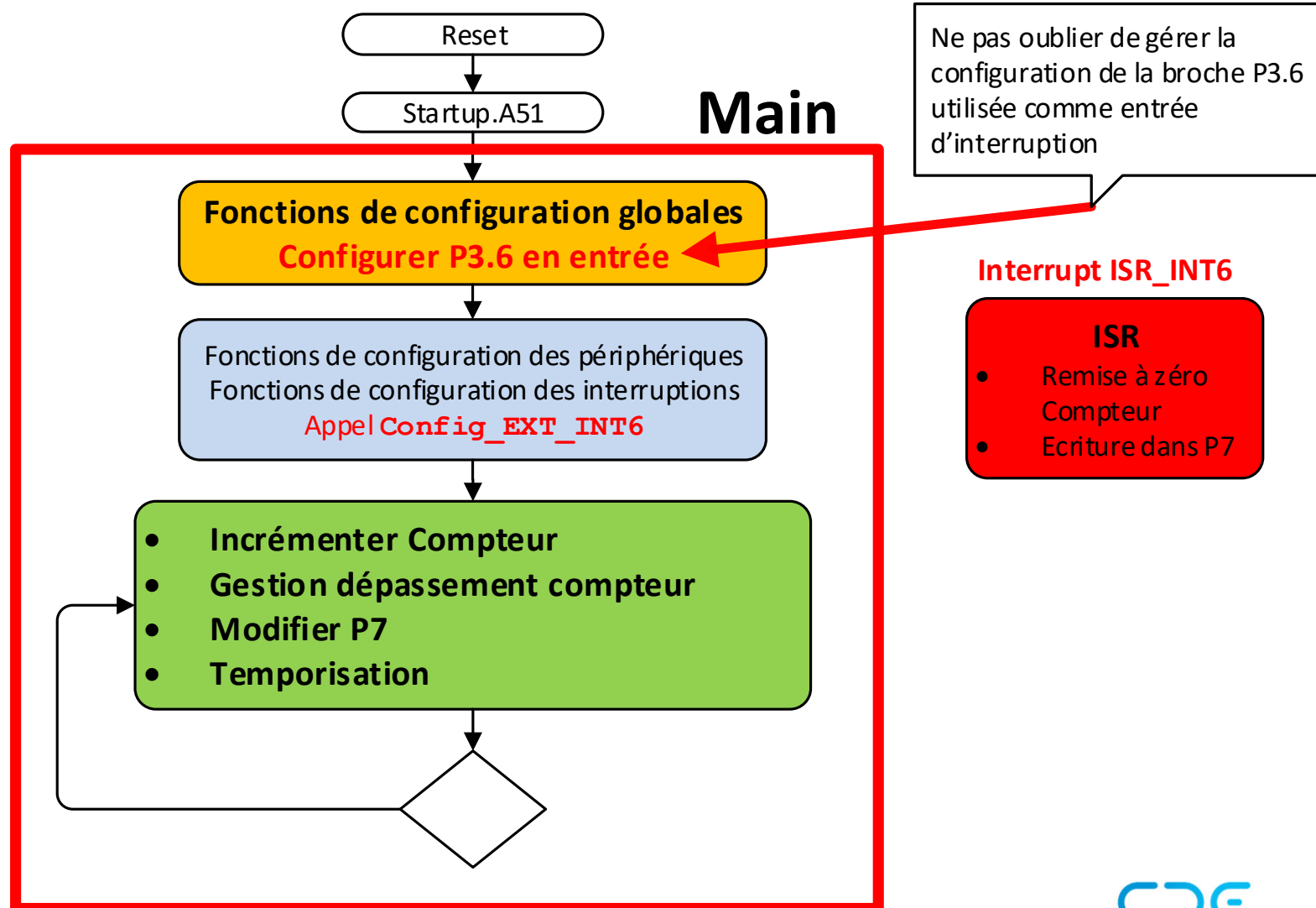
La variable `compteur_logiciel` est aussi manipulée dans le programme d'interruption.

Aussi, pour éviter tout risque « d'interférence » on interdit l'exécution des interruptions durant la manipulation de `compteur_logiciel` dans le « `main` »

On appelle cette portion de code, une section critique

Cette dévalidation des interruptions ne suspend pas leur détection, celles-ci seront exécutées dès la revalidation des interruptions.

Bilan mise en œuvre d'INT6



Activité 3 à 7 – Mise en œuvre des interruptions dans une application pratique

- Activité 3: Test du code sur la carte d'évaluation
- Activité 4: Ajout d'une interruption externe
- Activité 5: Gestion des priorités d'interruption
- Activité 6: Configuration des interruptions
- Activité 7: Mise en œuvre d'une interruption interne.



LIVE AND
DISCOVER

Contact

Nacer ABOUCHI
CPE Lyon

Damien FAVRE

François JOLY
CPE Lyon

Tél: 04 72 43 15 24
Nacer.abouchi@cpe.fr

damien.favre@cpe.fr

Tél. : 04 72 43 13 36
francois.joly@cpe.fr

www.cpe.fr